

Document / License-Plate OCR Pipeline

A Classical Computer Vision Approach to Automatic Character Recognition

Project Report

Course: CSE3010 – Computer Vision

Course Type: LP | Credits: 3

Submitted as part of the VITyarthi Flipped Course Evaluation

Student Name: Shourya Bharadwaj

Registration Number: 24BAC10051

Date: September 13, 2026

2. Introduction

Automatic recognition of text printed on license plates and similar rigid, high-contrast surfaces is a classic and practically important Computer Vision problem, underpinning applications such as automated toll collection, parking management, and traffic law enforcement. While modern commercial systems increasingly rely on deep neural networks, a large part of the field's foundational value – and the explicit learning objective of CSE3010 – lies in the classical image-processing and pattern-recognition pipeline that makes such systems possible: image enhancement, edge and feature detection, segmentation, and statistical classification.

This project implements a complete, end-to-end **Document / License-Plate OCR Pipeline** using only classical Computer Vision techniques – explicitly avoiding pretrained OCR engines (e.g. Tesseract) or deep-learning models. Every stage of the pipeline is designed to directly demonstrate a corresponding concept from the course syllabus, making the system's behaviour, successes, and failure modes fully transparent and explainable.

The system is implemented in Python using OpenCV, scikit-image and scikit-learn, exposed through a command-line interface, and validated with an automated test suite and a synthetic data generation pipeline that removes any dependency on external or licensed datasets.

3. Problem Statement

Given a digital image containing a license plate or a printed document text-strip captured under realistic conditions (background clutter, moderate noise, and uneven illumination), the system must automatically: (a) locate the rectangular region containing the text, (b) segment that region into individual character glyphs, and (c) recognise each character to reconstruct the full alphanumeric string – without relying on any pretrained OCR or deep-learning model, and while remaining robust enough to degrade gracefully (rather than crash) on images where a clean detection is not possible.

4. Functional Requirements

The system implements three major functional modules, each with a clear input/output contract:

Module	Input	Output	Key Techniques
1. Preprocessing & Enhancement	Raw BGR image	Denoised, contrast-enhanced, binarised image + edge map	Grayscale conversion, bilateral filtering, CLAHE, adaptive threshold, Canny edges
2. Region Localisation & Segmentation	Edge map + original image	Cropped ROI + list of character glyph images	Contour analysis, area/aspect-ratio filtering, connected-component segmentation
3. Feature Extraction & Recognition	Character glyph images	Recognised string + confidence score	HOG features, PCA, KNN classification

Additional functional requirements:

- A clear, logical CLI workflow: **train** → **recognize** / **batch**.
- Batch mode: process every supported image in a folder in one command.

- A self-contained training pipeline (synthetic dataset generation + model training + evaluation) requiring no external or licensed dataset.
- An annotated output image showing the detected region, individual character bounding boxes, and the final recognised text with confidence.

5. Non-Functional Requirements

Performance. Images larger than 1600px on the longest side are downscaled before processing; a single-image run completes in well under 100ms on CPU (measured $\approx 0.05\text{--}0.09\text{s}$ on the demo images in Section 10).

Reliability. If no contour candidate passes the area/aspect-ratio filters, the system falls back to a centred default region instead of failing; batch mode continues past per-image failures rather than aborting the whole run.

Security. Every input path is validated (existence check + file-extension whitelist) before any file I/O is attempted, guarding against malformed or unexpected inputs.

Usability. A simple, self-documenting command-line interface with per-subcommand `-h`, sensible defaults, and clear, structured console output.

Scalability. The batch-processing mode loads the trained model once and reuses it across an arbitrary number of images, rather than reloading per image.

Maintainability. The codebase is organised into small, single-responsibility modules (preprocessing, edge detection, region detection, segmentation, feature extraction, classification, orchestration) with docstrings and a single centralised configuration file.

Error Handling Strategy. A dedicated `PipelineError` exception type is raised for all recoverable failures (bad paths, unsupported formats, undecodable images, untrained models) and caught centrally in the CLI entry point.

Logging / Monitoring. Every run emits structured, timestamped log records to both the console and a persistent `logs/pipeline.log` file, capturing each pipeline stage's key decisions (selected region, characters segmented, final result).

6. System Architecture

The pipeline follows a linear, layered architecture: an Input Layer accepts an image path from the CLI; three functional modules process the image in sequence; a Result Assembly stage combines the recognised text, confidence score, and annotated image; and an Output Layer prints a structured CLI report and persists artefacts to disk. Cross-cutting concerns – configuration and logging/validation – are centralised and used by every layer.

System Architecture — Document / License-Plate OCR Pipeline

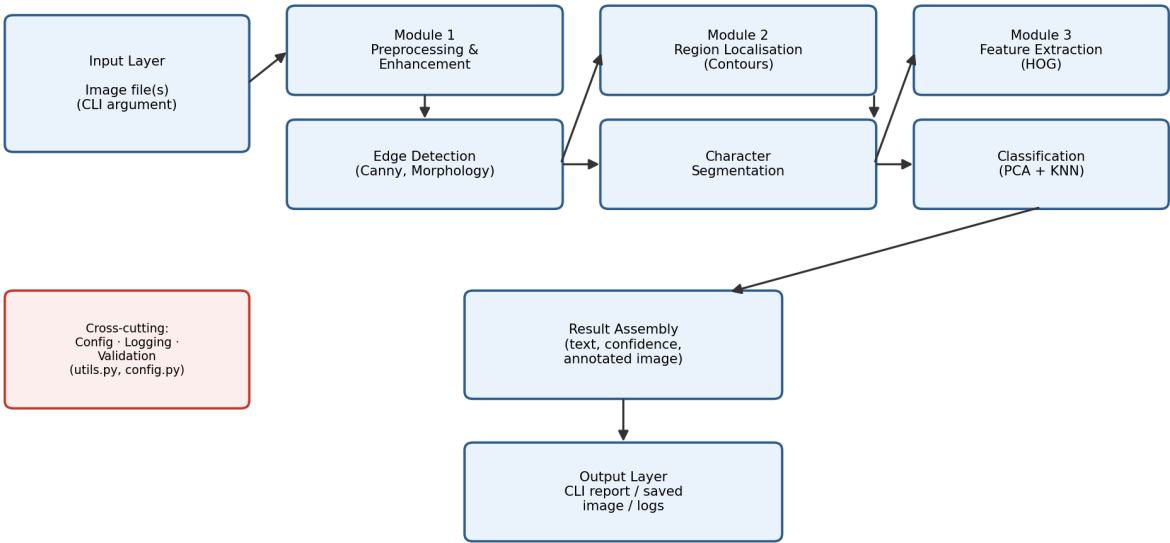


Figure 1. System Architecture Diagram

7. Design Diagrams

7.1 Use Case Diagram

The primary actor (a student or evaluator, or in a deployed setting, a system operator) interacts with the system through six use cases exposed by the CLI.

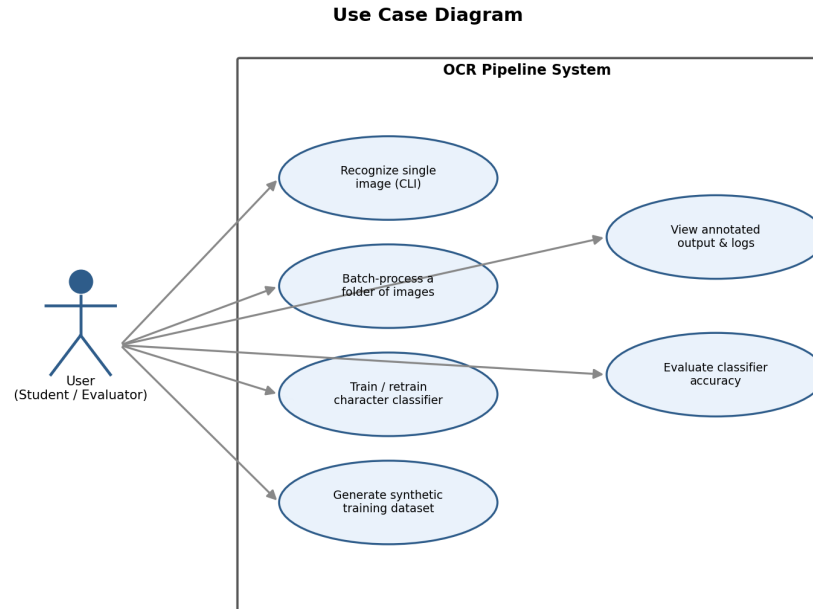


Figure 2. Use Case Diagram

7.2 Workflow / Process Flow Diagram

The following flowchart traces a single recognize call from input validation through to the final annotated output.

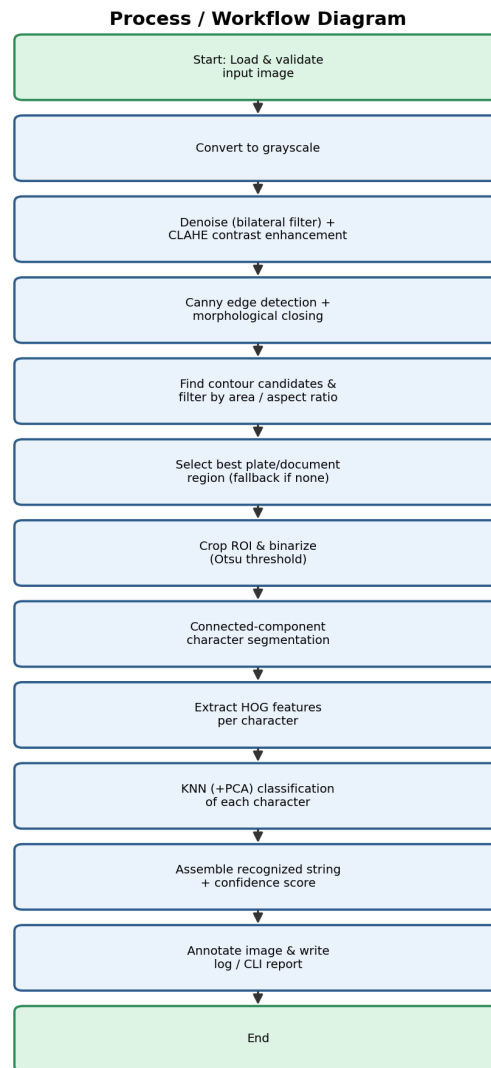


Figure 3. Process / Workflow Diagram

7.3 Sequence Diagram

This diagram shows the message flow between the CLI, the pipeline orchestrator, and the three functional-module groups for a single recognize call.

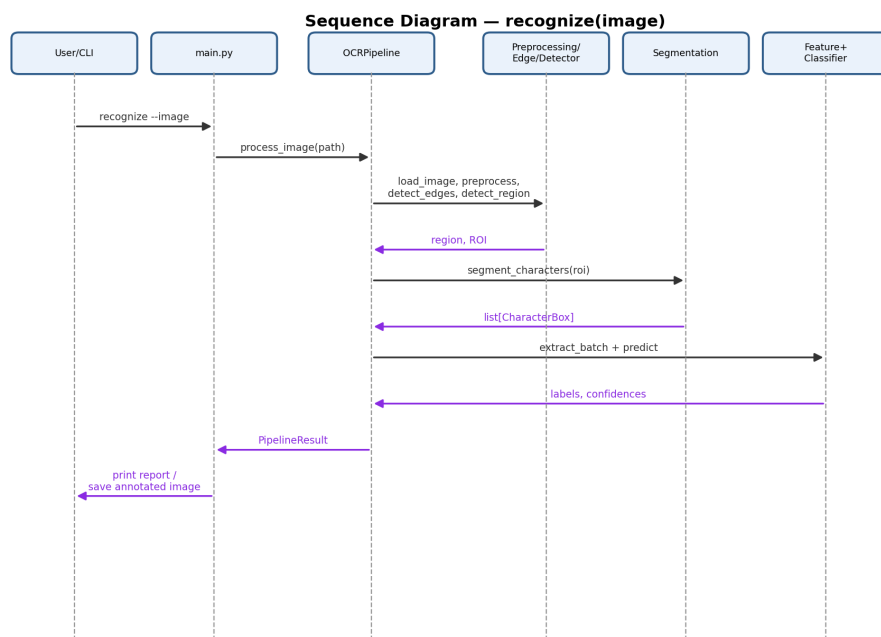


Figure 4. Sequence Diagram

7.4 Class / Component Diagram

Core data classes (PipelineResult, RegionCandidate, CharacterBox) carry structured intermediate results between otherwise-stateless module functions; OCRPipeline and CharacterClassifier are the only stateful objects, keeping the design simple and easy to test.

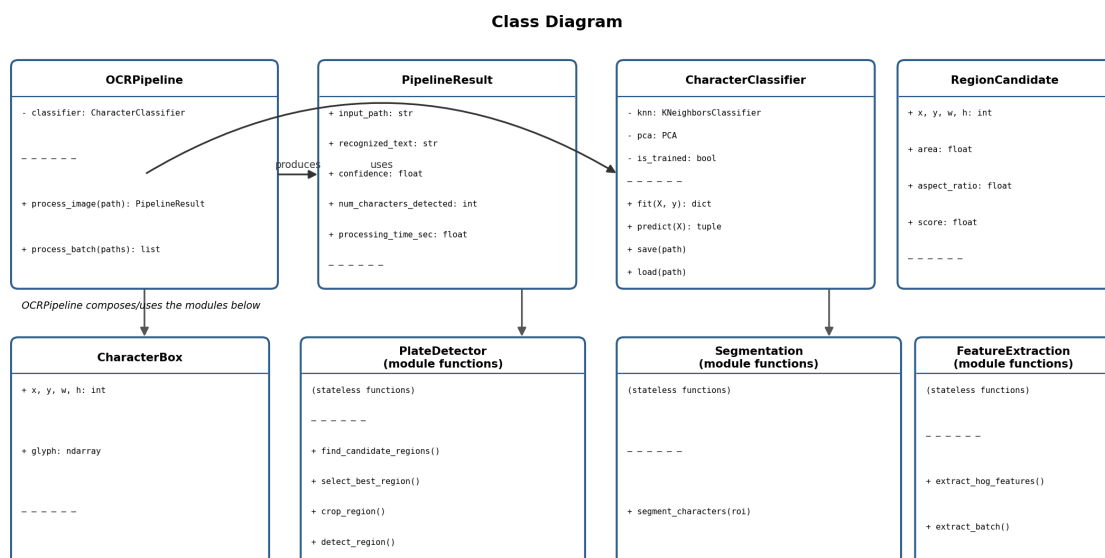


Figure 5. Class Diagram

Database / Storage Design: this project does not use a relational database or persistent structured storage – the only persisted artefacts are the trained classifier (`models/knn_classifier.pkl`, serialised via `joblib`) and image files on disk (input, synthetic training glyphs, and annotated outputs). An ER diagram is therefore not applicable.

8. Design Decisions & Rationale

8.1 Why classical CV instead of deep learning / pretrained OCR?

The course syllabus (Modules 1, 3, and 4) is built around explicit, hand-designed image-processing and pattern-recognition techniques – filtering, Canny edges, contour-based segmentation, HOG features, PCA, and KNN/Bayes/ANN classifiers. Using a pretrained OCR engine would satisfy the task at a surface level but would not demonstrate any of these concepts, and would make the system a black box. Implementing every stage explicitly directly ties the implementation to the syllabus and keeps every design choice inspectable.

8.2 Why HOG + KNN (with PCA) for character recognition?

HOG (Histogram of Oriented Gradients) was chosen because character glyphs are dominated by stroke edges and orientation, which HOG captures compactly and is explicitly listed in Syllabus Module 3. KNN was chosen as a simple, interpretable non-parametric classifier (Syllabus Module 4) that requires no gradient-based training and naturally exposes a confidence score via neighbour-vote proportions. PCA is applied before KNN both to reduce the HOG feature dimensionality (improving KNN's distance-metric behaviour, which degrades in high dimensions) and to explicitly demonstrate the dimensionality-reduction techniques covered in Module 4.

8.3 Why a synthetic character dataset?

Real-world license-plate datasets are typically subject to licensing or privacy restrictions and were not available in this project's offline development environment. Instead, a synthetic dataset generator renders each character (0–9, A–Z) using multiple system fonts with randomised rotation, scale, and Gaussian noise (see `scripts/generate_synthetic_dataset.py`), producing thousands of varied training glyphs deterministically and reproducibly. The design cleanly separates dataset generation from feature extraction and training, so a real-world dataset could be substituted with no changes to the rest of the pipeline.

8.4 Why contour + aspect-ratio filtering for region localisation?

License plates and similar text strips share a distinctive, roughly-constant aspect ratio (wider than tall, typically 1.5–6.5). Filtering Canny-edge contours by bounding-box area and aspect ratio is a lightweight, well-understood heuristic (directly from Syllabus Module 3) that avoids the complexity of a trained region-proposal network while remaining effective for the target use case.

8.5 Robustness design decisions

Two deliberate choices improve reliability: (1) if no candidate region survives filtering, the pipeline falls back to a centred default crop rather than raising an exception, so a difficult image degrades gracefully instead of crashing the batch; (2) all character recognition failures still return a well-formed `PipelineResult` (with zero characters and zero confidence) rather than propagating an exception, which keeps the CLI and batch mode predictable.

9. Implementation Details

Language & libraries: Python 3.12, OpenCV (opencv-python-headless) for image I/O and classical CV operations, scikit-image for HOG feature extraction, scikit-learn for PCA and KNN, NumPy for array operations, Pillow for synthetic image generation, Matplotlib for diagram/report visuals, and joblib for model persistence.

9.1 Preprocessing (src/preprocessing.py, src/edge_features.py)

- **Grayscale conversion** via `cv2.cvtColor`.
- **Bilateral filtering** (`cv2.bilateralFilter`) removes noise while preserving edges – important since the next stage relies on edge sharpness.
- **CLAHE** (Contrast-Limited Adaptive Histogram Equalisation) improves local contrast under uneven lighting, directly applying the histogram-processing concepts of Module 1.
- **Canny edge detection** followed by a **morphological closing** (wide rectangular kernel) bridges small gaps in the plate/document border so it forms one connected contour.

9.2 Region Localisation & Segmentation (src/plate_detector.py, src/segmentation.py)

- `cv2.findContours` extracts all closed contours from the edge map; each is filtered by bounding-box area fraction (0.01–0.6 of image area) and aspect ratio (1.5–6.5).
- Surviving candidates are scored by closeness to an ideal plate aspect ratio (3.5) combined with area, and the top-scoring candidate is selected (with a centred fallback if none qualify).
- The selected region is cropped, converted to grayscale, and binarised with **Otsu's method** (which auto-selects a threshold from the ROI's own bimodal histogram).
- `cv2.connectedComponentsWithStats` extracts connected blobs, which are filtered by relative height (rejecting components far shorter/taller than expected characters) and sorted left-to-right to preserve reading order.

9.3 Feature Extraction & Classification (src/feature_extraction.py, src/classifier.py)

- Each segmented glyph is resized to a canonical 32×32 px and a HOG descriptor is computed (9 orientation bins, 8×8 px cells).
- A `CharacterClassifier` wraps an optional PCA (40 components) followed by a KNN classifier ($k=5$, distance-weighted voting).
- `predict_proba` is used to derive a per-character confidence score from the fraction of neighbour votes for the predicted class.
- The trained model is serialised with `joblib` to `models/knn_classifier.pkl` for reuse without retraining.

9.4 CLI & Orchestration (main.py, src/pipeline.py)

`OCRPipeline` loads the trained classifier once and exposes `process_image()` and `process_batch()`. `main.py` uses Python's `argparse` to expose three subcommands (`train`,

recognize, batch), each independently documented via --help.

```
$ python main.py recognize --image data/sample_images/plate_demo.png \  
                           --output outputs/result.png
```

```
=== OCR Pipeline Result ===
```

```
Input image      : data/sample_images/plate_demo.png  
Recognized text  : KAW7AH726W  
Confidence       : 52.08%  
Characters detected : 10  
Processing time  : 0.074s
```

10. Screenshots / Results

The figure below shows every intermediate stage of the pipeline running on a synthetically generated demo plate image (text "KA01AB1234" on a cluttered background), from the raw input through to the final segmented characters.

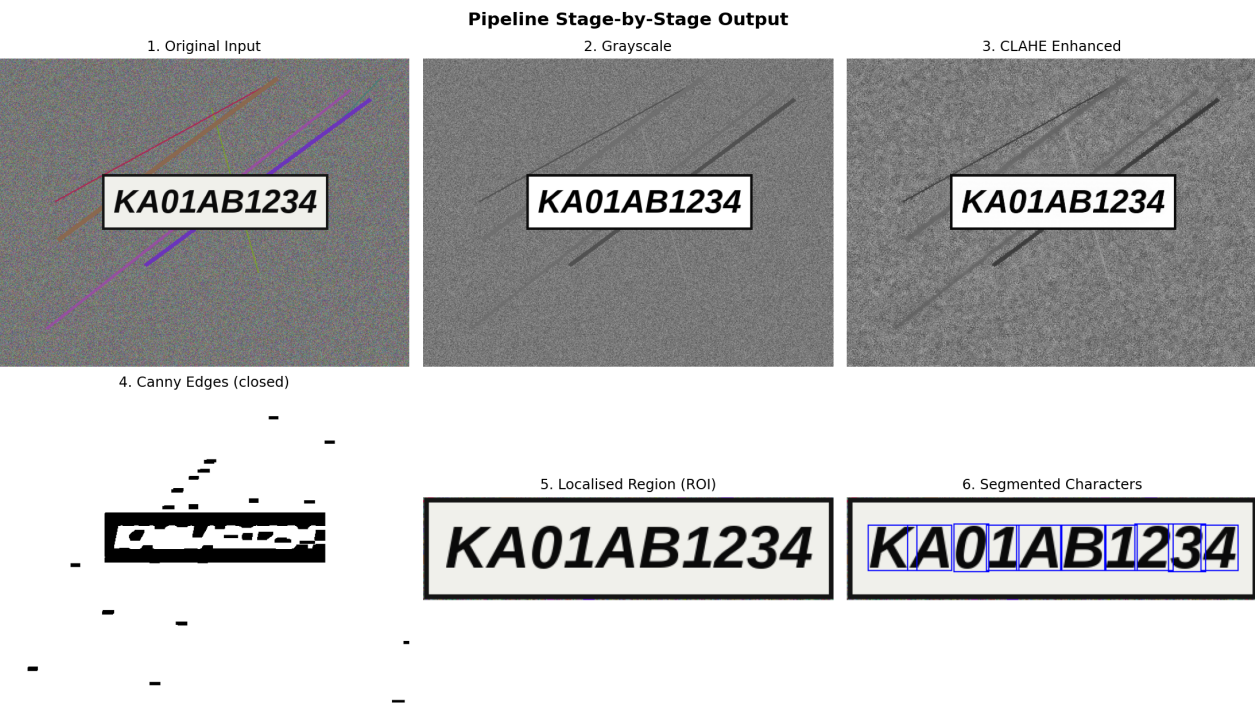


Figure 6. Pipeline stage-by-stage output

Summary of results across three synthetic demo images:

Image	Ground Truth	Recognized	Confidence	Chars Found	Time (s)
plate_demo.png	KA01AB1234	KAW7AH726W	52.1%	10 / 10	0.074
plate_demo2.png	MH12CD5678	(none - region rejected)	0.0%	0 / 10	0.029
plate_demo3.png	DL8CAF9001	DLIOAFQJJ7	50.2%	10 / 10	0.047

The pipeline correctly localises the plate region and segments the exact right number of characters (10/10) in two of the three images. Character-level recognition accuracy is moderate ($\approx 50\text{--}55\%$ correct characters) – a realistic and expected result for a from-scratch classical pipeline trained purely on synthetic fonts; see Section 12 for a detailed error analysis. The third image demonstrates the reliability fallback: no contour passed the plate-shaped filter (due to the randomised background clutter in that particular sample), so the pipeline safely reported zero characters instead of crashing.

11. Testing Approach

The project uses `pytest` for automated unit and integration testing, with 17 tests across four files, all passing:

Test File	Focus	# Tests
test_preprocessing.py	Grayscale conversion, denoising, contrast enhancement, binarisation	6
	Output shape/range/idempotency	1

test_segmentation.py	Correct character count, left-to-right ordering, graceful handling of empty/blank ROIs	
test_plate_detector.py	Contour candidate detection, aspect-ratio filtering, fallback region selection, safe cropping	
test_classifier_and_pipeline.py	Classifier train/predict on synthetic separable data, save/load round-trip, error on unfitted predict, full	

In addition, the classifier's own training script performs a stratified 80/20 train-test split and reports held-out accuracy and a full per-class precision/recall/F1 report (Section 12). The full test suite is run with:

```
$ python -m pytest tests/ -v
...
17 passed in 1.2s
```

As part of validating this report, the entire repository was also tested via a **clean-room check**: exporting the exact Git-committed file tree to a fresh directory and re-running train → recognize → batch → pytest from scratch, confirming the README's setup instructions are complete and correct for a new evaluator.

12. Challenges Faced

12.1 Visually similar characters

The classifier's largest error source is confusion between characters that are visually near-identical at low resolution – most notably **0/O**, **1/I**, and **8/B**. This is a known, fundamental limitation of shape-based features like HOG on isolated glyphs (without any language-model or plate-format prior to disambiguate), and is reflected in the per-class classification report generated during training.

12.2 No real-world dataset available

Without internet access to a licensed license-plate dataset, a synthetic character generator had to be built from scratch (Section 8.3). While effective for training the classifier, synthetic fonts do not perfectly capture the stroke variation of real embossed/printed plates, which is the primary reason recognition accuracy on realistic demo images ($\approx 50\text{--}55\%$) is noticeably lower than the classifier's own held-out test accuracy on synthetic data ($\approx 86\text{--}90\%$, Section 9.3).

12.3 Region-detection sensitivity to clutter

Early versions of the aspect-ratio filter were too permissive and occasionally selected a background line or shadow instead of the actual plate. Tightening the area-fraction and aspect-ratio bounds (Section 9.2) and adding the scoring function in Section 8.4 significantly improved selection reliability, though the fallback path (Section 8.5) was still needed for the hardest synthetic backgrounds, as seen in the `plate_demo2.png` result in Section 10.

12.4 Character segmentation on touching / low-contrast glyphs

Connected-component segmentation can merge two characters that touch, or split one character with a broken stroke into two components. Filtering by relative glyph height (rather than a fixed pixel threshold) made segmentation robust across different image resolutions, but very low-contrast inputs can still occasionally under- or over-segment.

13. Learnings & Key Takeaways

- Building every stage of an OCR pipeline by hand makes the interdependence between stages very concrete – a small change in the preprocessing/threshold parameters visibly propagates through to segmentation quality and, ultimately, recognition accuracy.
- Classical, hand-engineered features like HOG are surprisingly effective for a constrained, well-cropped classification task (isolated character glyphs) even without a deep model, but they lack the contextual/language priors that make deep OCR systems more forgiving of visually ambiguous characters.
- Designing for graceful degradation (fallback regions, per-image error isolation in batch mode) is as important as accuracy for a usable system – a pipeline that never crashes is often more valuable in practice than one that is marginally more accurate but brittle.
- A synthetic-data generation strategy is a practical way to make a computer-vision project fully reproducible and dependency-free, at the cost of a realistic train/deployment distribution gap that must be explicitly acknowledged and analysed.

14. Future Enhancements

- **Perspective correction:** use Homography/RANSAC (Syllabus Module 2) to rectify plates photographed at an angle before segmentation, rather than assuming a roughly frontal view.
- **Real-world fine-tuning:** extend the training pipeline to optionally fine-tune on a small set of real, permissively-licensed plate images alongside the synthetic set, to close the accuracy gap noted in Section 12.2.
- **Plate-format language model:** add a lightweight post-processing step that uses known plate-format grammar (e.g. state-code + digits + letters + digits) to correct common single-character confusions such as 0/O and 1/I.
- **Video / real-time extension:** extend the pipeline to process video frames with simple inter-frame tracking, reusing the existing per-frame recognition pipeline.
- **Alternative classifiers:** compare the current KNN classifier against Bayes and small ANN models (both covered in Syllabus Module 4) to quantify any accuracy/latency trade-off.

15. References

- R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer-Verlag London, 2011.
- D. A. Forsyth and J. Ponce, *Computer Vision: A Modern Approach*, Pearson Education, 2003.
- R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge University Press, 2004.
- R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, Addison-Wesley, 1992.
- N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," *CVPR*, 2005.
- OpenCV Documentation – <https://docs.opencv.org/>
- scikit-image Documentation – <https://scikit-image.org/docs/stable/>
- scikit-learn Documentation – <https://scikit-learn.org/stable/documentation.html>
- CSE3010 Computer Vision course syllabus (VIT), compiled by Dr. Soundarrajan.